INTRODUCTION OF INTELLIGENT COMPUTING CHIP AND SYSTEM

LABORATORY REPORT

Lab 01: Backpropagation and Three-Linear-Layer Neural Network Classification on MNIST

Student Name: Adeyinka Kehibde

Student ID: 2960012

Course: Introduction of Intelligent Computing Chip and System

Instructor: Shiwei Liu, Assistant Professor

Laboratory: Lab 01

Dataset: MNIST

Date: October 7, 2026

Abstract

This laboratory experiment investigates both the theoretical derivation and practical implementation of backpropagation in a three-linear-layer neural network. First, the backpropagation equation for weight W_3 is derived using the chain rule. Second, a fully connected neural network is implemented in PyTorch and trained for handwritten-digit classification using the MNIST dataset.

The implemented network follows the architecture

$$784 \to 256 \to 128 \to 10.$$

The original MNIST training data are separated into training and validation subsets, while the official 10,000-image test set is used for final evaluation. Training loss, validation loss, training accuracy, and validation accuracy are monitored over ten epochs. The final performance of the trained model is reported using overall classification accuracy on the independent MNIST test dataset.

1 Introduction

Artificial neural networks learn by progressively modifying their trainable parameters in order to minimize an objective or loss function. Backpropagation is the principal algorithm used to calculate the gradients required for this parameter optimization.

During forward propagation, input data move through the neural network to produce predictions. A loss function then quantifies the difference between these predictions and the corresponding ground-truth labels.

During backward propagation, the chain rule of calculus is applied through the computational graph in reverse order. This determines how strongly each weight contributes to the final prediction error.

The objectives of this laboratory experiment are:

1. Derive the backpropagation equation for weight W_3.
2. Construct a neural network containing exactly three linear layers.
3. Train the neural network using the MNIST handwritten-digit dataset.
4. Monitor the training and validation loss over multiple epochs.
5. Monitor training and validation classification accuracy.
6. Evaluate the final trained network using the independent MNIST test dataset.
7. Report the overall test classification accuracy.

2 Backpropagation Derivation for W_3

Consider the neural-network structure discussed in the lecture. Weight W_3 connects input I_2 to hidden neuron H_1.

The pre-activation value of the first hidden neuron can be expressed as

$$H_1 = I_1 W_1 + I_2 W_3 + b_1. \tag{1}$$

Assuming that a sigmoid activation function is applied,

$$HA_1 = \sigma(H_1) = \frac{1}{1 + e^{-H_1}}. \tag{2}$$

The derivative of the sigmoid activation is

$$\frac{\partial HA_1}{\partial H_1} = HA_1(1 - HA_1). \tag{3}$$

2.1 Loss Function

For the two-output example considered in the theoretical derivation, the mean-squared-error loss is

$$e = \frac{1}{n}\left[(y_1 - OA_1)^2 + (y_2 - OA_2)^2\right]. \tag{4}$$

The derivative of the loss with respect to OA_1 is

$$\frac{\partial e}{\partial OA_1} = \frac{2}{n}(OA_1 - y_1). \tag{5}$$

Similarly,

$$\frac{\partial e}{\partial OA_2} = \frac{2}{n}(OA_2 - y_2). \tag{6}$$

2.2 Chain-Rule Derivation for W_3

Weight W_3 affects hidden neuron H_1, and therefore influences both output neurons through the activation HA_1.

Consequently,

$$\begin{aligned}\frac{\partial e}{\partial W_3} =& \frac{\partial e}{\partial OA_1}\frac{\partial OA_1}{\partial O_1}\frac{\partial O_1}{\partial HA_1}\frac{\partial HA_1}{\partial H_1}\frac{\partial H_1}{\partial W_3} \\ &+ \frac{\partial e}{\partial OA_2}\frac{\partial OA_2}{\partial O_2}\frac{\partial O_2}{\partial HA_1}\frac{\partial HA_1}{\partial H_1}\frac{\partial H_1}{\partial W_3}.\end{aligned} \tag{7}$$

From Equation 1,

$$\boxed{\frac{\partial H_1}{\partial W_3} = I_2} \tag{8}$$

For the second layer,

$$\frac{\partial O_1}{\partial HA_1} = W_7 \tag{9}$$

and

$$\frac{\partial O_2}{\partial HA_1} = W_8. \tag{10}$$

Substitution into the chain-rule expression gives

$$\boxed{\frac{\partial e}{\partial W_3} = \frac{2}{n} I_2 HA_1(1 - HA_1)\left[(OA_1 - y_1)W_7 + (OA_2 - y_2)W_8\right]} \tag{11}$$

Equation 11 represents the required backpropagation gradient for weight W_3.

2.3 Weight Update

After calculating the gradient, the parameter can be updated using gradient descent:

$$\boxed{W_3^{(t+1)} = W_3^{(t)} - \eta \frac{\partial e}{\partial W_3}} \tag{12}$$

where η denotes the learning rate.

3 MNIST Dataset

The MNIST dataset contains grayscale images of handwritten digits from 0 to 9.

Each image has dimensions

$$28 \times 28$$

pixels.

Consequently, each image contains

$$28 \times 28 = 784 \tag{13}$$

pixel values.

For this experiment, the original 60,000-image MNIST training partition is divided into

- 55,000 training images;
- 5,000 validation images.

The official MNIST test partition contains 10,000 independent images and is used for the final model evaluation.

Before training, the images are transformed into PyTorch tensors and normalized according to

$$x_{\text{norm}} = \frac{x - \mu}{\sigma}, \tag{14}$$

where

$$\mu = 0.1307$$

and

$$\sigma = 0.3081.$$

4 Three-Linear-Layer Neural Network

Each 28×28 input image is first flattened into a 784-dimensional feature vector.

The implemented architecture is

$$\boxed{784 \to 256 \to 128 \to 10} \tag{15}$$

Therefore, the three linear transformations are

$$\text{Linear Layer 1:} \quad 784 \to 256, \tag{16}$$

$$\text{Linear Layer 2:} \quad 256 \to 128, \tag{17}$$

$$\text{Linear Layer 3:} \quad 128 \to 10. \tag{18}$$

The first and second layers are followed by the rectified linear unit (ReLU) activation function:

$$\mathrm{ReLU}(x) = \max(0, x). \tag{19}$$

The third linear layer produces ten output logits corresponding to the ten MNIST digit classes.

No explicit softmax layer is added because PyTorch's `CrossEntropyLoss` accepts the raw output logits directly.

5 Classification Loss

The experimental model uses categorical cross-entropy loss.

For a mini-batch of B samples and $C = 10$ digit classes,

$$\mathcal{L} = -\frac{1}{B} \sum_{i=1}^{B} \sum_{c=1}^{C} y_{ic} \log(p_{ic}), \tag{20}$$

where

- y_{ic} represents the target class indicator;
- p_{ic} represents the predicted probability for class c;
- B represents the mini-batch size.

6 Training Procedure

The neural network is trained for ten epochs using mini-batch optimization.

The principal experimental settings are:

- Batch size: 128;
- Number of epochs: 10;
- Learning rate: 0.001;
- Optimizer: Adam;
- Loss function: CrossEntropyLoss;
- Random seed: 42.

For each mini-batch, the following procedure is performed:

1. Clear gradients from the previous iteration.
2. Perform forward propagation.
3. Calculate the classification loss.
4. Perform backpropagation.
5. Update the trainable weights.

The central PyTorch training operations are

```
optimizer.zero_grad()
outputs = model(images)
loss = criterion(outputs, labels)
loss.backward()
optimizer.step()
```

The command

```
loss.backward()
```

performs automatic differentiation and computes the gradients of the loss function with respect to all trainable parameters.

7 Evaluation Metric

Overall classification accuracy is calculated as

$$\boxed{\text{Accuracy} = \frac{N_{\text{correct}}}{N_{\text{total}}} \times 100\%} \tag{21}$$

where N_{correct} represents the number of correctly classified images and N_{total} represents the total number of evaluated test images.

8 Training and Validation Loss

Figure 1 presents the measured training and validation cross-entropy loss across the ten training epochs.

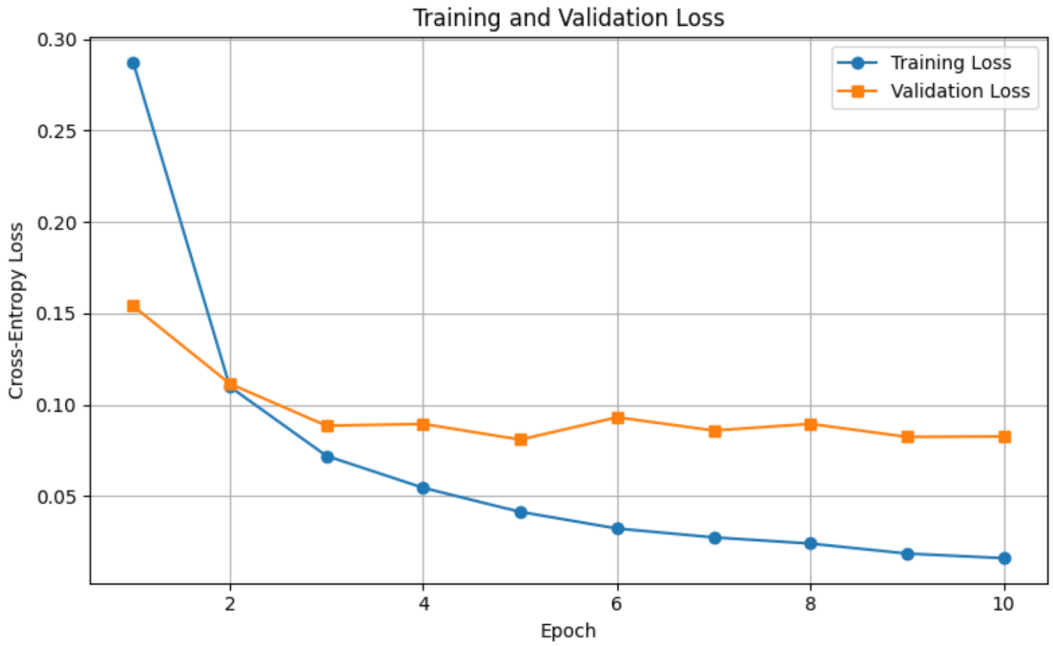


Figure 1: Training and validation cross-entropy loss over ten epochs.

The training loss decreases substantially from approximately 0.29 during the first epoch to below 0.02 by the final epoch. This progressive decline indicates that the optimization procedure successfully minimizes the classification objective.

The validation loss also decreases strongly during the early stages of training, falling from approximately 0.15 to around 0.09 during the first several epochs. After this initial reduction, the validation loss fluctuates within a relatively narrow region.

The difference between the continuously decreasing training loss and the more stable validation loss indicates that additional training mainly improves the fit to the training dataset after the initial epochs. Nevertheless, validation performance remains stable and does not exhibit a large sustained degradation.

9 Training and Validation Accuracy

Figure 2 presents the measured training and validation classification accuracy.

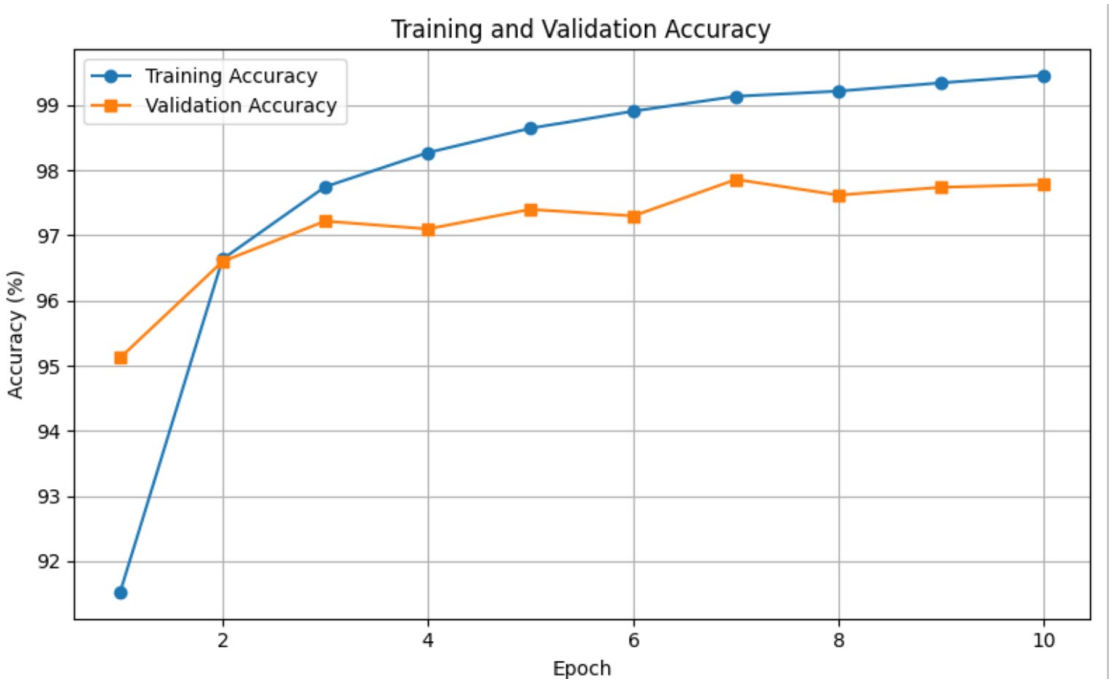


Figure 2: Training and validation classification accuracy over ten epochs.

Training accuracy increases rapidly during the first several epochs. It rises from approximately 91.5% after the first epoch to more than 96% after the second epoch and exceeds 99% during the later stages of training.

Validation accuracy similarly improves rapidly during the first several epochs. It begins at approximately 95% and subsequently stabilizes in the high-97% range.

The relatively small gap between training and validation accuracy during the early stages indicates good initial generalization. A larger gap develops during later epochs as training accuracy continues to increase while validation accuracy becomes comparatively stable. This is consistent with mild overfitting, although validation performance itself remains high.

10 Overall Test Accuracy

After completion of training, the model is evaluated using all 10,000 samples in the independent MNIST test dataset.

The final overall test accuracy obtained from the Google Colab experiment is

$$\boxed{\text{Overall Test Accuracy} = \quad 97.95\%} \tag{22}$$

The final test accuracy represents the proportion of previously unseen MNIST images that were classified correctly by the trained three-linear-layer neural network.

In the final rerun, 9,795 of 10,000 test images were classified correctly (97.95%).

11 Discussion

The experimental results demonstrate that a relatively simple three-linear-layer fully connected neural network can achieve strong classification performance on the MNIST handwritten-digit dataset.

The loss curve in Figure 1 confirms that the optimization algorithm successfully minimizes the training objective. Most of the reduction occurs during the first few epochs, after which the training loss continues to decrease more gradually.

The validation loss behaves differently from the training loss. Although it decreases substantially during the early training stages, it later oscillates around a relatively stable value. This behavior suggests that the largest improvement in generalization occurs during the early epochs.

The accuracy curve in Figure 2 provides additional evidence of successful learning. Training accuracy increases consistently and exceeds 99% during the later epochs. Validation accuracy reaches approximately the high-97% range and then remains relatively stable.

The growing difference between training and validation performance in the later epochs indicates that the model increasingly specializes to the training data. However, because validation accuracy remains high rather than collapsing, the observed behavior is more appropriately described as mild overfitting rather than severe overfitting.

The experimental design also preserves a separate test dataset. Training samples are used to calculate parameter gradients, validation samples are used to observe generalization during model development, and the official MNIST test dataset is used only for final evaluation.

A convolutional neural network was deliberately not implemented. Although CNNs are generally more suitable for exploiting the spatial structure of images, the objective of this laboratory experiment is specifically to train a neural network containing three linear layers. The selected model therefore directly follows the assigned requirement.

12 Relationship Between Mathematical Backpropagation and PyTorch

The theoretical derivation of W_3 demonstrates explicitly how the gradient of the loss is obtained through successive applications of the chain rule.

In the PyTorch implementation, the same principle is carried out using automatic differentiation.

During the forward pass,

```
outputs = model(images)
loss = criterion(outputs, labels)
```

PyTorch records the operations required to construct the loss.

During the backward pass,

```
loss.backward()
```

the computational graph is traversed in reverse order and the gradient of the loss with respect to each trainable parameter is calculated.

The optimizer then executes

```
optimizer.step()
```

to update the parameters using the calculated gradients.

Therefore, the manual derivation of

$$\frac{\partial e}{\partial W_3}$$

and PyTorch's automatic differentiation mechanism represent the same underlying mathematical principle.

13 Conclusion

This laboratory experiment investigated neural-network backpropagation both analytically and experimentally.

First, the gradient for weight W_3 was obtained using the chain rule:

$$\boxed{\frac{\partial e}{\partial W_3} = \frac{2}{n} I_2 H A_1 (1 - H A_1) \left[(O A_1 - y_1) W_7 + (O A_2 - y_2) W_8 \right]} \tag{23}$$

The corresponding weight update is

$$W_3^{(t+1)} = W_3^{(t)} - \eta \frac{\partial e}{\partial W_3}. \tag{24}$$

Second, a three-linear-layer neural network with architecture

$$\boxed{784 \to 256 \to 128 \to 10} \tag{25}$$

was implemented using PyTorch and trained using the MNIST handwritten-digit dataset.

The experimental curves demonstrate successful convergence. Training loss decreased consistently, while training accuracy exceeded 99%. Validation accuracy stabilized in the high-97% range, indicating strong generalization despite a modest training-validation gap during later epochs.

Finally, the trained network was evaluated on the independent MNIST test dataset and obtained an overall test accuracy of

$$\boxed{97.95\%} \tag{26}$$

The experiment therefore establishes a clear relationship between the analytical chain-rule derivation of backpropagation and its practical implementation through PyTorch automatic differentiation.

14 Appendix: Complete Python Implementation

The following Python code was used to train the three-linear-layer neural network and generate the experimental results shown in this report.

```

# ================================================================
# LAB 01
# Three-Linear-Layer Neural Network for MNIST Classification
#
# Student: Adeyinka Kehibde
# Student ID: 2960012
# Course: Introduction of Intelligent Computing Chip and System
# Instructor: Shiwei Liu, Assistant Professor
# Date: October 7, 2026
# ================================================================

import random
import numpy as np
import matplotlib.pyplot as plt

import torch
import torch.nn as nn
import torch.optim as optim

from torchvision import datasets, transforms
from torch.utils.data import DataLoader, random_split


# ================================================================
# REPRODUCIBILITY
# ================================================================

SEED = 42

random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)

if torch.cuda.is_available():
    torch.cuda.manual_seed(SEED)
    torch.cuda.manual_seed_all(SEED)

torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False


```

```
# ================================================================
# DEVICE
# ================================================================

device = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)

print("=" * 70)
print("LAB 01: THREE-LINEAR-LAYER NEURAL NETWORK")
print("=" * 70)

print("Student Name : Adeyinka Kehibde")
print("Student ID   : 2960012")
print("Device       :", device)

if torch.cuda.is_available():
    print("GPU          :", torch.cuda.get_device_name(0))

print("=" * 70)


# ================================================================
# EXPERIMENTAL PARAMETERS
# ================================================================

INPUT_SIZE = 28 * 28

HIDDEN_SIZE_1 = 256
HIDDEN_SIZE_2 = 128

NUM_CLASSES = 10

BATCH_SIZE = 128
LEARNING_RATE = 0.001
NUM_EPOCHS = 10

TRAIN_SIZE = 55000
VALIDATION_SIZE = 5000


# ================================================================
# DATA PREPROCESSING
# ================================================================

transform = transforms.Compose([

```

```
    transforms.ToTensor(),

    transforms.Normalize(
        (0.1307,),
        (0.3081,)
    )

])


# ================================================================
# LOAD MNIST
# ================================================================

full_training_dataset = datasets.MNIST(

    root="./data",

    train=True,

    download=True,

    transform=transform

)


test_dataset = datasets.MNIST(

    root="./data",

    train=False,

    download=True,

    transform=transform

)


# ================================================================
# TRAINING AND VALIDATION SPLIT
# ================================================================

train_dataset, validation_dataset = random_split(

    full_training_dataset,
```

```
137
138     [TRAIN_SIZE, VALIDATION_SIZE],
139
140     generator=torch.Generator().manual_seed(SEED)
141
142 )
143
144
145 # ================================================================
146 # DATA LOADERS
147 # ================================================================
148
149 train_loader = DataLoader(
150
151     train_dataset,
152
153     batch_size=BATCH_SIZE,
154
155     shuffle=True
156
157 )
158
159
160 validation_loader = DataLoader(
161
162     validation_dataset,
163
164     batch_size=BATCH_SIZE,
165
166     shuffle=False
167
168 )
169
170
171 test_loader = DataLoader(
172
173     test_dataset,
174
175     batch_size=BATCH_SIZE,
176
177     shuffle=False
178
179 )
180
181
182 # ================================================================
183 # THREE-LINEAR-LAYER MODEL
```

```
184 # ================================================================
185
186 class ThreeLinearNN(nn.Module):
187
188     def __init__(self):
189
190         super().__init__()
191
192
193         # Linear Layer 1
194         self.fc1 = nn.Linear(
195             INPUT_SIZE,
196             HIDDEN_SIZE_1
197         )
198
199
200         # Linear Layer 2
201         self.fc2 = nn.Linear(
202             HIDDEN_SIZE_1,
203             HIDDEN_SIZE_2
204         )
205
206
207         # Linear Layer 3
208         self.fc3 = nn.Linear(
209             HIDDEN_SIZE_2,
210             NUM_CLASSES
211         )
212
213
214         self.relu = nn.ReLU()
215
216
217     def forward(self, x):
218
219         # Flatten 28 x 28 image into 784 features
220         x = torch.flatten(
221             x,
222             start_dim=1
223         )
224
225
226         # First linear layer
227         x = self.fc1(x)
228         x = self.relu(x)
229
230
```

```
        # Second linear layer
        x = self.fc2(x)
        x = self.relu(x)


        # Third linear layer
        x = self.fc3(x)


        return x


# ================================================================
# CREATE MODEL
# ================================================================

model = ThreeLinearNN().to(device)

print("\nMODEL ARCHITECTURE")
print(model)


# ================================================================
# LOSS FUNCTION AND OPTIMIZER
# ================================================================

criterion = nn.CrossEntropyLoss()


optimizer = optim.Adam(

    model.parameters(),

    lr=LEARNING_RATE

)


# ================================================================
# EVALUATION FUNCTION
# ================================================================

def evaluate(model, data_loader):

    model.eval()

    total_loss = 0.0
```

```
278
279     correct = 0
280
281     total = 0
282
283
284     with torch.no_grad():
285
286         for images, labels in data_loader:
287
288
289             images = images.to(device)
290
291             labels = labels.to(device)
292
293
294             outputs = model(images)
295
296
297             loss = criterion(
298                 outputs,
299                 labels
300             )
301
302
303             total_loss += (
304                 loss.item()
305                 * labels.size(0)
306             )
307
308
309             predictions = torch.argmax(
310                 outputs,
311                 dim=1
312             )
313
314
315             correct += (
316                 predictions == labels
317             ).sum().item()
318
319
320             total += labels.size(0)
321
322
323     average_loss = (
324         total_loss / total
```

```
    )


    accuracy = (
        100.0
        * correct
        / total
    )


    return average_loss, accuracy


# ================================================================
# STORE RESULTS
# ================================================================

training_losses = []

validation_losses = []

training_accuracies = []

validation_accuracies = []


# ================================================================
# TRAINING
# ================================================================

print("\nTRAINING STARTED")

print("=" * 70)


for epoch in range(NUM_EPOCHS):


    model.train()


    running_loss = 0.0

    correct = 0

    total = 0

```

```
372
373     for images, labels in train_loader:
374
375
376         images = images.to(device)
377
378         labels = labels.to(device)
379
380
381         # Clear previous gradients
382         optimizer.zero_grad()
383
384
385         # Forward propagation
386         outputs = model(images)
387
388
389         # Calculate loss
390         loss = criterion(
391             outputs,
392             labels
393         )
394
395
396         # Backpropagation
397         loss.backward()
398
399
400         # Update model parameters
401         optimizer.step()
402
403
404         running_loss += (
405             loss.item()
406             * labels.size(0)
407         )
408
409
410         predictions = torch.argmax(
411             outputs,
412             dim=1
413         )
414
415
416         correct += (
417             predictions == labels
418         ).sum().item()
```

```
419
420
421             total += labels.size(0)
422
423
424         # ------------------------------------------------------------
425         # TRAINING RESULTS
426         # ------------------------------------------------------------
427
428         training_loss = (
429             running_loss / total
430         )
431
432
433         training_accuracy = (
434             100.0
435             * correct
436             / total
437         )
438
439
440         # ------------------------------------------------------------
441         # VALIDATION RESULTS
442         # ------------------------------------------------------------
443
444         validation_loss, validation_accuracy = evaluate(
445
446             model,
447
448             validation_loader
449
450         )
451
452
453         # ------------------------------------------------------------
454         # SAVE RESULTS FOR GRAPHS
455         # ------------------------------------------------------------
456
457         training_losses.append(
458             training_loss
459         )
460
461
462         validation_losses.append(
463             validation_loss
464         )
465
```

```
466
467     training_accuracies.append(
468         training_accuracy
469     )
470
471
472     validation_accuracies.append(
473         validation_accuracy
474     )
475
476
477     print(
478
479         f"Epoch {epoch + 1:02d}/{NUM_EPOCHS} | "
480
481         f"Train Accuracy: {training_accuracy:.2f}% | "
482
483         f"Validation Accuracy: {validation_accuracy:.2f}%"
484
485     )
486
487
488 print("=" * 70)
489
490 print("TRAINING COMPLETED")
491
492
493 # ======================================================================
494 # FINAL TEST EVALUATION
495 # ======================================================================
496
497 test_loss, test_accuracy = evaluate(
498
499     model,
500
501     test_loader
502
503 )
504
505
506 # ======================================================================
507 # GRAPH 1 - TRAINING AND VALIDATION LOSS
508 # ======================================================================
509
510 epochs = range(
511     1,
512     NUM_EPOCHS + 1
```

```
)


plt.figure(
    figsize=(8, 5)
)


plt.plot(

    epochs,

    training_losses,

    marker="o",

    label="Training Loss"

)


plt.plot(

    epochs,

    validation_losses,

    marker="s",

    label="Validation Loss"

)


plt.xlabel("Epoch")

plt.ylabel("Cross-Entropy Loss")

plt.title(
    "Training and Validation Loss"
)

plt.legend()

plt.grid(True)

plt.tight_layout()
```

```
560
561 plt.savefig(
562     "g1.png",
563     dpi=300,
564     bbox_inches="tight"
565 )
566
567 plt.show()
568
569
570 # ================================================================
571 # GRAPH 2 - TRAINING AND VALIDATION ACCURACY
572 # ================================================================
573
574 plt.figure(
575     figsize=(8, 5)
576 )
577
578
579 plt.plot(
580
581     epochs,
582
583     training_accuracies,
584
585     marker="o",
586
587     label="Training Accuracy"
588
589 )
590
591
592 plt.plot(
593
594     epochs,
595
596     validation_accuracies,
597
598     marker="s",
599
600     label="Validation Accuracy"
601
602 )
603
604
605 plt.xlabel("Epoch")
606
```

```
plt.ylabel("Accuracy (%)")

plt.title(
    "Training and Validation Accuracy"
)

plt.legend()

plt.grid(True)

plt.tight_layout()

plt.savefig(
    "g2.jpg",
    dpi=300,
    bbox_inches="tight"
)

plt.show()


# ================================================================
# FINAL OVERALL ACCURACY
# ================================================================

print("\n")

print("=" * 70)

print("FINAL MNIST RESULT")

print("=" * 70)


print(
    f"Overall Test Accuracy: "
    f"{test_accuracy:.2f}%"
)


print("=" * 70)
```

Listing 1: PyTorch implementation of the three-linear-layer MNIST classifier.

15 References

1. PyTorch, *PyTorch Documentation*. `https://pytorch.org/docs/stable/index.html`

2. Y. LeCun, C. Cortes, and C. J. C. Burges, *The MNIST Database of Handwritten Digits*.